KiriliX

Практически Наръчник

Примери · Грешки · Поведение на Езика

- Синтаксис и типове
- Масиви и функции
- Всички типове грешки
- Примерни програми
- Капани и неочаквано поведение

Легенда на Цветовете

В целия наръчник са използвани следните цветови кодове:

Код (.bg файл)

```
// Син блок = изходен код на BgLangC
цяло x = 5;
печатай(x);
```

Изход (конзола)

```
// Зелен блок = успешен изход
5
```

```
// Само син блок = код без изход (декларирание, теория)
```

```
Грешка: Червен блок = Runtime или синтактична грешка
```

```
Внимание: Оранжев блок = предупреждение, неочаквано поведение
```

1. Основен Синтаксис и Типове

1.1 Декларираме на Променливи

Декларацията изисква тип пред името. Ако не зададете стойност, се ползва стойност по подразбиране.

```
цяло брой = 42;
дробно пи = 3.14;
текст поздрав = "Здравей, свят!";
булево флаг = вярно;
символ буква = 'A';
динамично нещо = 100;
```

Декларираме без начална стойност — всеки тип получава нулева стойност:

Код

```
цяло брой;
дробно число;
текст низ;
булево флаг;
печатай(брой);
печатай(число);
печатай(низ);
печатай(флаг);
```

Изход

```
0
0.000000

грешно
```

1.2 Цели Числа (цяло)

Код

```
цяло а = 10;
цяло б = 3;
печатай(а + б);
печатай(а - б);
печатай(а * б);
печатай(а / б);    // Целочислено!
печатай(а % б);
```

Изход

```
13
7
30
3
1
```

Важно: Делението на две цели числа винаги дава цяло (`10 / 3 = 3`, не 3.333). Остатъкът се губи.

1.3 Дробни Числа (дробно)

Код

```
дробно x = 10.0;
дробно y = 3.0;
печатай(x / y);

// Смесване: цяло + дробно
```

Изход

```
3.333333



12.500000
```

```
цяло а = 10;
дробно б = 2.5;
печатай(а + б);
печатай(а * б);
```

```
25.000000
```

Забележка: Когато смесвате цяло и дробно в операция, цялото автоматично се преобразува към дробно.

1.4 Текст (текст)

Код

```
текст а = "Здравей, ";
текст б = "свят!";
печатай(а + б);

// Конкатенация с числа
цяло год = 2026;
печатай("Година: " + към_текст(год));

// Дължина на текст
текст дума = "BgLangC";
печатай(дължина(дума));
```

Изход

```
Здравей, свят!


Година: 2026


7
```

1.5 Булеви Стойности (булево)

Код

```
булево а = вярно;
булево б = грешно;

печатай(а && б);
печатай(а || б);
печатай(а == б);
печатай(а != б);

// Сравнения дават булево
цяло х = 5;
печатай(х > 3);
печатай(х == 10);
```

Изход

```
грешно
вярно
грешно
вярно

вярно
грешно
```

1.6 Символи (символ)

Код

```
символ буква = 'А';
символ цифра = '7';
символ нов_ред = '\n';
символ табулация = '\t';
символ апостроф = '\'';
символ наклонена = '\\';

печатай(буква);
печатай(цифра);
печатай(апостроф);
```

Изход

```
А
7
'
```

1.7 Динамичен Тип (динамично)

Единственият тип, позволяващ смяна на стойността с друг тип по всяко време:

Код

```
динамично x = 42;
печатай(x);

x = "вече съм текст";
печатай(x);

x = [1, 2, 3];
печатай(x);

x = вярно;
печатай(x);

x = нищо;
печатай(x);
```

Изход

```
42

вече съм текст

[1, 2, 3]

вярно

нищо
```

2. Масиви

2.1 Създаване и Достъп

Код
```
цяло[] числа = [10, 20, 30, 40, 50];

печатай(числа);
печатай(числа[0]);
печатай(числа[4]);
печатай(дължина(числа));
```

Изход
```
[10, 20, 30, 40, 50]
10
50
5
```

2.2 Промяна на Елементи

Код
```
цяло[] масив = [1, 2, 3];
печатай(масив);

масив[0] = 99;
масив[2] = 77;
печатай(масив);
```

Изход
```
[1, 2, 3]

[99, 2, 77]
```

2.3 нов_масив() — Масив с Фиксиран Размер

Код
```
цяло[] буфер = нов_масив(5);
печатай(буфер);

буфер[0] = 100;
буфер[3] = 777;
печатай(буфер);
```

Изход
```
[0, 0, 0, 0, 0]

[100, 0, 0, 777, 0]
```

2.4 Различни Типове Масиви

```
текст[] думи = ["едно", "две", "три"];
булево[] флагове = [вярно, грешно, вярно];
дробно[] стойности = [1.1, 2.2, 3.3];

// Динамичен масив — смесени типове
динамично смесен = [1, "текст", вярно, 3.14];
```

Код
```
текст[] думи = ["А", "Б", "В"];
печатай(думи[1]);
думи[0] = "Новото";
печатай(думи);

динамично смесен = [1, "два", вярно];
печатай(смесен);
```

Изход
```
Б
["Новото", "Б", "В"]

[1, "два", вярно]
3.140000
```

```
смесен[0] = 3.14;
печатай(смесен[0]);
```

2.5 Итерация Върху Масив

Код

```
цяло[] числа = [10, 20, 30, 40];
цяло дълж = дължина(числа);

за (цяло и = 0; и < дълж; и = и + 1) {
    печатай("[" + към_текст(и) + "] = "
            + към_текст(числа[и]));
}
```

Изход

```
[0] = 10
[1] = 20
[2] = 30
[3] = 40
```

3. Контролни Структури

3.1 Условие ако / иначе

Код
```
цяло x = 15;

ако (x > 10) {
    печатай("По-голямо от 10");
} иначе {
    печатай("10 или по-малко");
}
```

Изход
```
По-голямо от 10
```

Верига от условия (else-if):

Код
```
цяло оценка = 75;

ако (оценка >= 90) {
    печатай("Отличен");
} иначе {
    ако (оценка >= 75) {
        печатай("Много добър");
    } иначе {
        ако (оценка >= 60) {
            печатай("Добър");
        } иначе {
            печатай("Слаб");
        }
    }
}
```

Изход
```
Много добър
```

3.2 Цикъл докато

Код
```
цяло i = 1;
докато (i <= 5) {
    печатай(i);
    i = i + 1;
}
```

Изход
```
1
2
3
4
5
```

Натрупване на сума:

Код
```
цяло сума = 0;
цяло i = 1;
докато (i <= 100) {
    сума = сума + i;
    i = i + 1;
}
печатай("Сума 1..100 = " +
към_текст(сума));
```

Изход
```
Сума 1..100 = 5050
```

3.3 Цикъл за

Код

```
за (цяло j = 0; j < 5; j = j + 1) {
    печатай(j * j);
}
```

Изход

```
0
1
4
9
16
```

FizzBuzz — класически пример:

Код

```
за (цяло и = 1; и <= 15; и = и + 1) {
    ако (и % 15 == 0) {
        печатай("FizzBuzz");
    } иначе {
        ако (и % 3 == 0) {
            печатай("Fizz");
        } иначе {
            ако (и % 5 == 0) {
                печатай("Buzz");
            } иначе {
                печатай(и);
            }
        }
    }
}
```

Изход

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

4. Функции

4.1 Дефиниция и Извикване

Код

```
функция цяло събери(цяло а, цяло б) {
    върни а + б;
}

функция текст поздрав(текст име) {
    върни "Здравей, " + име + "!";
}

функция празно главно() {
    печатай(събери(3, 7));
    печатай(поздрав("Иван"));
    върни нищо;
}
```

Изход

```
10
Здравей, Иван!
```

4.2 Void Функции (празно)

Код

```
функция празно нарисувай_линия(цяло дълж)
{
    цяло i = 0;
    текст линия = "";
    докато (i < дълж) {
        линия = линия + "-";
        i = i + 1;
    }
    печатай(линия);
    върни нищо;
}

функция празно главно() {
    нарисувай_линия(10);
    нарисувай_линия(5);
    върни нищо;
}
```

Изход

```
----------
-----
```

4.3 Рекурсия — Факториел

Код

```
функция цяло факт(цяло н) {
    ако (н <= 1) {
        върни 1;
    }
    върни н * факт(н - 1);
}

функция празно главно() {
    за (цяло i=1; i<=7; i=i+1) {
        печатай(към_текст(i) + "! = "
                + към_текст(факт(i)));
    }
    върни нищо;
```

Изход

```
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
```

```
}
```

4.4 Рекурсия — Фибоначи

Код

```
функция цяло фиб(цяло н) {
    ако (н <= 1) { върни н; }
    върни фиб(н-1) + фиб(н-2);
}

функция празно главно() {
    за (цяло i=0; i<=10; i=i+1) {
        печатай("fib(" + към_текст(i)
                + ") = " +
към_текст(фиб(i)));
    }
    върни нищо;
}
```

Изход

```
fib(0) = 0
fib(1) = 1
fib(2) = 1
fib(3) = 2
fib(4) = 3
fib(5) = 5
fib(6) = 8
fib(7) = 13
fib(8) = 21
fib(9) = 34
fib(10) = 55
```

4.5 Ханойски Кули

Код

```
функция празно ханой(цяло д,
    текст от, текст към, текст пом) {
    ако (д == 1) {
        печатай("Диск 1: " + от + "→" +
към);
        върни нищо;
    }
    ханой(д-1, от, пом, към);
    печатай("Диск " + към_текст(д)
            + ": " + от + "→" + към);
    ханой(д-1, пом, към, от);
    върни нищо;
}
функция празно главно() {
    ханой(3, "A", "C", "B");
    върни нищо;
}
```

Изход

```
Диск 1: A→C
Диск 2: A→B
Диск 1: C→B
Диск 3: A→C
Диск 1: B→A
Диск 2: B→C
Диск 1: A→C
```

5. Стандартна Библиотека — Примери

5.1 към_текст() и към_цяло()

Код

```
цяло а = 42;
дробно б = 3.14;
булево в = вярно;

печатай(към_текст(а));
печатай(към_текст(б));
печатай(към_текст(в));

// Обратно:
текст числен = "123";
цяло резултат = към_цяло(числен);
печатай(резултат + 1);
```

Изход

```
42
3.140000
вярно


124
```

5.2 дължина()

Код

```
текст дума = "Здравей";
цяло[] числа = [1,2,3,4,5];

печатай(дължина(дума));
печатай(дължина(числа));
```

Изход

```
7
5
```

5.3 корен() и степен()

Код

```
печатай(корен(16));
печатай(корен(2));
печатай(степен(2, 10));
печатай(степен(3, 3));
```

Изход

```
4.000000
1.414214
1024.000000
27.000000
```

5.4 начало() и край() — Измерване на Скорост

Код

```
функция цяло фиб(цяло н) {
    ако (н <= 1) { върни н; }
    върни фиб(н-1) + фиб(н-2);
}

функция празно главно() {
    начало();
    цяло р = фиб(30);
    дробно т = край();
    печатай("fib(30) = " + към_текст(р));
    печатай("Изминало: " + към_текст(т) +
" сек");
    върни нищо;
}
```

Примерен изход

```
fib(30) = 832040
Изминало: 0.231000 сек
```

6. Математически Операции и Приоритет

6.1 Всички Оператори

Код

```
цяло а = 10;
цяло б = 3;
дробно в = 2.5;

печатай(а + б);      // 13
печатай(а - б);      // 7
печатай(а * б);      // 30
печатай(а / б);      // 3  (цяло!)
печатай(а % б);      // 1
печатай(а / в);      // 4.000000
печатай(а + б * в); // приоритет: * преди
+
печатай((а+б) * в); // скоби
```

Изход

```
13
7
30
3
1
4.000000
17.500000
32.500000
```

6.2 Логически Оператори и Short-Circuit

Код

```
булево а = вярно;
булево б = грешно;

// Short-circuit && : ако левият е
грешно,
// десният ИЗОБЩО не се оценява
печатай(б && а);  // грешно (спира при б)
печатай(а || б);  // вярно  (спира при а)

// Сравнения:
цяло х = 5;
печатай(х > 3 && х < 10);
печатай(х == 5 || х == 6);
печатай(х != 5);
```

Изход

```
грешно
вярно
вярно
вярно
грешно
```

7. Грешки — Пълен Каталог

Тази глава покрива всяка категория грешка в BgLangC — с конкретен код, точното съобщение и обяснение.

7.1 Типова Несъвместимост (Runtime)

Опит за записване на стойност от грешен тип в строго типизирана променлива:

```
// ГРЕШКА: Записваме текст в цяло число
цяло брой = 42;
брой = "осемдесет";  // <-- ТУК
```

```
Грешка: Неуспешно присвояване на 'брой'. Променливата може да не съществува
или типовете да не съвпадат.
```

```
// ГРЕШКА: Записваме дробно в булево
булево флаг = вярно;
флаг = 3.14;  // <-- ТУК
```

```
Грешка: Неуспешно присвояване на 'флаг'. Променливата може да не съществува
или типовете да не съвпадат.
```

```
// ГРЕШКА: Записваме символ в текст
текст дума = "Здравей";
дума = 'А';  // символ != текст
```

```
Грешка: Неуспешно присвояване на 'дума'. Променливата може да не съществува
или типовете да не съвпадат.
```

Решение: Използвайте `динамично` ако стойността трябва да може да сменя тип, или конвертирайте с `към_текст()` / `към_цяло()`.

7.2 Недефинирана Променлива (Runtime)

```
функция празно главно() {
    цяло а = 5;
    цяло резултат = а + б;  // 'б' никога не е дефинирана!
    печатай(резултат);
    върни нищо;
}
```

```
Грешка: Променливата 'б' не е дефинирана.
```

Решение: Декларирайте `цяло б = 0;` преди да я ползвате.

7.3 Повторна Декларация в Един Scope (Runtime)

```
функция празно главно() {
    цяло x = 10;
    цяло x = 20;  // 'x' вече съществува в този scope!
    печатай(x);
    върни нищо;
}
```

```
Грешка: Променливата 'x' вече е дефинирана.
```

Забележка: Засенчването (shadowing) е позволено само в **различни блокове**. В един и същи блок — грешка.

7.4 Извикване на Несъществуваща Функция (Runtime)

```
функция празно главно() {
    цяло p = сортирай([3,1,2]);  // 'сортирай' не е дефинирана!
    печатай(p);
    върни нищо;
}
```

```
Грешка: Функцията 'сортирай' не е намерена.
```

7.5 Грешен Брой Аргументи (Runtime)

```
функция цяло събери(цяло а, цяло б) {
    върни а + б;
}

функция празно главно() {
    // Функцията иска 2 аргумента, подаваме само 1
    цяло p = събери(5);
    печатай(p);
    върни нищо;
}
```

```
Грешка: Невалиден брой аргументи за 'събери'.
```

```
// Или пък твърде много:
цяло р = събери(5, 10, 15);  // 3 вместо 2
```

```
Грешка: Невалиден брой аргументи за 'събери'.
```

7.6 Индекс Извън Границите на Масив (Runtime)

```
функция празно главно() {
    цяло[] списък = [10, 20, 30];
    // Масивът има индекси 0, 1, 2 — опитваме 4
    цяло стойност = списък[4];
    печатай(стойност);
    върни нищо;
}
```

```
Грешка: Индекс 4 извън границите (размер: 3).
```

```
// Отрицателен индекс също е невалиден:
цяло лошо = списък[-1];
```

```
Грешка: Индекс -1 извън границите при присвояване.
```

7.7 Индексиране на Не-Масив (Runtime)

```
функция празно главно() {
    цяло число = 42;
    цяло лошо = число[0];  // число не е масив!
    печатай(лошо);
    върни нищо;
}
```

```
Грешка: Опит за достъп по индекс на не-масив.
```

7.8 Грешен Тип Елемент в Типизиран Масив (Runtime)

```
функция празно главно() {
    цяло[] числа = [1, 2, 3];
    числа[1] = "текст";  // масивът е цяло[], не текст[]!
    печатай(числа);
    върни нищо;
}
```

```
Грешка: Типът на елемента не съвпада със строго-типизирания масив!
```

7.9 Синтактични Грешки (Parse-Time)

Синтактичните грешки се хващат преди изпълнение — при парсването:

```
// Забравена точка-и-запетая
цяло x = 10
цяло y = 20;
```

```
Синтактична грешка (ред 2): syntax error близо до 'цяло'
```

```
// Неотворена скоба
ако x > 5) {
    печатай("ok");
}
```

```
Синтактична грешка (ред 1): syntax error близо до 'x'
```

```
// Неизвестен символ
цяло @ = 5;
```

```
Грешка (ред 1): Неочакван символ '@'
Синтактична грешка (ред 1): syntax error близо до '='
```

7.10 Небулево Условие в ако / докато (Тихо поведение)

```
// Условието трябва да е булево!
// В C 'if (5)' минава, тук НЕ.
цяло число = 5;
ако (число) {            // число е цяло, не булево
    печатай("влязохме");
}
```

```
Програмата не крашва, но блокът НЕ се изпълнява.
Условието оценява до 'нищо' (VAL_NULL), което не е VAL_BOOL + true,
затова клонът се пропуска мълчаливо.
```

Решение: Използвайте явно сравнение: `ако (число != 0)` или `ако (число > 0)`.

7.11 Функция без върни (Тихо поведение)

```
функция цяло събери(цяло а, цяло б) {
```

```
    цяло резултат = а + б;
    // Забравихме: върни резултат;
}

функция празно главно() {
    цяло отговор = събери(5, 10);
    печатай(отговор);
    върни нищо;
}
```

```
Не се хвърля грешка, но:
- събери() връща нищо (VAL_NULL)
- присвояването цяло отговор = нищо може да доведе до непредвидимо поведение
```

Правило: Всяка не-`празно` функция трябва да завършва с `върни стойност;`.

7.12 Повторна Декларация на Функция (Runtime)

```
функция цяло пресметни(цяло x) {
    върни x * 2;
}

// Дефинираме я отново!
функция цяло пресметни(цяло x) {
    върни x * 3;
}
```

```
Грешка: Функцията 'пресметни' вече е дефинирана.
```

7.13 Безкрайна Рекурсия (Stack Overflow)

Рекурсия без базово условие изчерпва стека на операционната система:

```
функция празно безкрайност(цяло стъпка) {
    // Няма базово условие!
    печатай("Дълбочина: " + към_текст(стъпка));
    безкрайност(стъпка + 1);
    върни нищо;
}

функция празно главно() {
    безкрайност(1);
    върни нищо;
}
```

```
Дълбочина: 1
Дълбочина: 2
...
Дълбочина: 261917
Segmentation fault (core dumped)
```

Правило: Всяка рекурсивна функция трябва да има **базово условие** (`ако (н <= 1) { върни ...; }`) което спира рекурсията.

7.14 Грешки в Стандартната Библиотека

дължина() с неправилен тип:

```
дължина(42);  // работи само с текст и масиви
Грешка: дължина() работи само с текст и масиви.
```

корен() с текст:

```
корен("пет");  // работи само с числа
Грешка: корен() работи само с числа.
```

нов_масив() с отрицателен размер:

```
нов_масив(-5);  // невалиден размер
Грешка: нов_масив() не може да има отрицателен размер.
```

нов_масив() без аргумент от тип цяло:

```
нов_масив(3.14);  // трябва цяло
Грешка: нов_масив() очаква 1 аргумент от тип цяло.
```

8. Капани и Неочаквано Поведение

8.1 Целочислено Деление

Код
```
// КАПАН: Деление на две цели
цяло а = 7;
цяло б = 2;
печатай(а / б);   // Очаквате 3.5?

// ПРАВИЛНО: Поне един дробен
дробно в = 7.0;
печатай(в / б);

// Или принудителна конверсия:
дробно р = а / б;  // пак 3 !
печатай(р);
```

Изход
```
3
3.500000
3.000000
```

Урок: Ако искате дробен резултат от деление, поне един от операндите трябва да е **дробно**.

8.2 Модификация на Масив вътре във Функция

Масивите се предават по стойност — копира се структурата, но елементите са на heap. Модификацията НА ЕЛЕМЕНТ реално засяга оригинала:

Код
```
функция празно добави_десет(цяло масив) {
    цяло дълж = дължина(масив);
    за (цяло i=0; i<дълж; i=i+1) {
        масив[i] = масив[i] + 10;
    }
    върни нищо;
}

функция празно главно() {
    цяло числа = [1, 2, 3];
    печатай(числа);
    добави_десет(числа);
    печатай(числа); // Променен!
    върни нищо;
}
```

Изход
```
[1, 2, 3]
[11, 12, 13]
```

8.3 Засенчване на Променливи (Shadowing)

Код
```
цяло x = 100;

функция празно главно() {
    печатай(x);  // 100 — глобалното

    цяло x = 5;  // Засенчва глобалното
    печатай(x);  // 5 — локалното
```

Изход
```
100
5
99
5
```

```
    ако (вярно) {
        цяло x = 99; // Засенчва
локалното
        печатай(x);  // 99 — вложеното
    }

    печатай(x);  // 5 — локалното пак
    върни нищо;
}
```

8.4 Конкатенация: Трябва Изричен към_текст()

```
// ГРЕШКА: + между текст и число без конверсия
цяло брой = 5;
печатай("Броят е: " + брой);  // ГРЕШКА!
```

```
Програмата не знае как да събере текст и цяло директно.
```

```
// ПРАВИЛНО:
печатай("Броят е: " + към_текст(брой));


// Или:
текст съобщение = "Броят е: " + към_текст(брой);
печатай(съобщение);
```

```
Броят е: 5
```

8.5 Функциите Виждат Само Глобалния Scope

Код

```
цяло глобална = 42;

функция празно покажи() {
    // Вижда глобалната променлива
    печатай(глобална);
    върни нищо;
}

функция празно главно() {
    цяло локална = 99;

    // покажи() НЕ вижда 'локална'!
    // Функциите имат достъп само до
    // глобалния scope + собствените
    // параметри
    покажи();
    върни нищо;
}
```

Изход

```
42
```

9. Пълни Примерни Програми

9.1 Bubble Sort

Код

```
функция празно сортирай(цяло масив) {
    цяло дл = дължина(масив);
    булево разменени = вярно;
    докато (разменени) {
        разменени = грешно;
        за (цяло i=0; i<дл-1; i=i+1) {
            ако (масив[i] > масив[i+1]) {
                цяло т = масив[i];
                масив[i] = масив[i+1];
                масив[i+1] = т;
                разменени = вярно;
            }
        }
    }
    върни нищо;
}
функция празно главно() {
    цяло числа = [64,34,25,12,22,11,90];
    печатай(числа);
    сортирай(числа);
    печатай(числа);
    върни нищо;
}
```

Изход

```
[64, 34, 25, 12, 22, 11, 90]
[11, 12, 22, 25, 34, 64, 90]
```

9.2 Прости Числа

Код

```
функция празно главно() {
    цяло лимит = 30;
    за (цяло н=2; н<=лимит; н=н+1) {
        булево просто = вярно;
        за (цяло д=2; д<=н/2; д=д+1) {
            ако (н % д == 0) {
                просто = грешно;
            }
        }
        ако (просто) {
            печатай(н);
        }
    }
    върни нищо;
}
```

Изход

```
2
3
5
7
11
13
17
19
23
29
```

9.3 Намиране на Максимум в Масив

Код

```
функция цяло намери_макс(цяло масив) {
    цяло макс = масив[0];
    цяло дл = дължина(масив);
    за (цяло i=1; i<дл; i=i+1) {
        ако (масив[i] > макс) {
            макс = масив[i];
```

Изход

```
Максимум: 99
```

```
        }
    }
    върни макс;
}
функция празно главно() {
    цяло числа = [3, 77, 12, 5, 44, 99,
8];
    цяло макс = намери_макс(числа);
    печатай("Максимум: " +
към_текст(макс));
    върни нищо;
}
```

9.4 Степен — Итеративна Версия

Код

```
функция цяло степен_цяло(цяло основа,
цяло степ) {
    цяло резултат = 1;
    за (цяло i=0; i<степ; i=i+1) {
        резултат = резултат * основа;
    }
    върни резултат;
}
функция празно главно() {
    печатай(степен_цяло(2, 0));
    печатай(степен_цяло(2, 8));
    печатай(степен_цяло(3, 5));
    печатай(степен_цяло(10, 4));
    върни нищо;
}
```

Изход

```
1
256
243
10000
```

9.5 Обръщане на Масив

Код

```
функция празно обърни(цяло масив) {
    цяло ляво = 0;
    цяло дясно = дължина(масив) - 1;
    докато (ляво < дясно) {
        цяло т = масив[ляво];
        масив[ляво] = масив[дясно];
        масив[дясно] = т;
        ляво = ляво + 1;
        дясно = дясно - 1;
    }
    върни нищо;
}
функция празно главно() {
    цяло числа = [1,2,3,4,5];
    печатай(числа);
    обърни(числа);
    печатай(числа);
    върни нищо;
}
```

Изход

```
[1, 2, 3, 4, 5]
[5, 4, 3, 2, 1]
```

9.6 Таблица за Умножение

Код

Изход (частичен)

```
функция празно главно() {
    за (цяло i=1; i<=5; i=i+1) {
        за (цяло j=1; j<=5; j=j+1) {
            цяло p = i * j;
            // Няма форматиране, само
печатай
            печатай(към_текст(i) + "x"
               + към_текст(j) + "="
               + към_текст(p));
        }
    }
    върни нищо;
}
```

```
1x1=1
1x2=2
1x3=3
...
5x4=20
5x5=25
```

10. Бърза Справка — Шпаргалка

10.1 Типове

```
цяло      n = 42;
дробно    x = 3.14;
текст     s = "Здравей";
булево    b = вярно;
символ    c = 'A';
динамично d = нищо;   // Може да бъде всичко

цяло[]    масив   = [1, 2, 3];
текст[]   думи    = ["а", "б"];
динамично смесен  = [1, "две", вярно];
```

10.2 Оператори

```
// Аритметични
+  -  *  /  %

// Сравнение
==  !=  <  >  <=  >=

// Логически
&&  ||

// Присвояване
=
```

10.3 Контролни Структури

```
// Условие
ако (условие) { ... } иначе { ... }

// While цикъл
докато (условие) { ... }

// For цикъл
за (цяло i=0; i<10; i=i+1) { ... }

// Функция
функция тип_връщане ime(тип параметър) { ... }
функция празно главно() { ... }
```

10.4 Вградени Функции

```
печатай(x)                // Принтира стойност + нов ред
дължина(масив_или_текст) // Брой елементи/символи
корен(x)                  // sqrt(x) -> дробно
степен(основа, степен)    // pow() -> дробно
към_текст(x)              // Конвертира всичко -> текст
към_цяло(x)               // Конвертира текст/дробно -> цяло
вход()                    // Чете ред от stdin -> текст
нов_масив(n)              // Нов масив с n елемента
начало()                  // Старт на хронометър
край()                    // Стоп + връща секунди (дробно)
```

10.5 Честите Грешки — Ремайндер

- **TypeMismatch:** Присвоявате стойност от грешен тип → ползвайте `динамично` или конвертирайте.
- **Undefined:** Достъпвате недефинирана променлива → декларирайте я преди употреба.
- **OutOfBounds:** Индексът излиза от 0..дължина-1 → проверете с `дължина()`.
- **Wrong args:** Различен брой аргументи → проверете сигнатурата на функцията.
- **No return:** Не-`празно` функция без `върни` → добавете `върни стойност;`.
- **Integer division:** 10/3 = 3, не 3.33 → ползвайте `дробно` ако искате десетична точност.
- **Concatenation:** "Текст" + 5 е грешка → `"Текст" + към_текст(5)`.
- **Stack overflow:** Рекурсия без базово условие → добавете `ако (n<=0) { върни ...; }`.

BgLangC — Програмирай на Родния Език